XC企业版 - 源程序鉴别材料
软件著作权补正上传专用
软件名称：XC企业版
版本号：V9.0
生成日期：2026年7月4日
==========

文件：前端源代码.txt
行数：44675
==========

```
// 文件：App.vue
<script setup>
import { onBeforeUnmount, onMounted, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import MainLayout from './components/MainLayout.vue'
import ProMode from './components/ProMode.vue'
const route = useRoute()
const router = useRouter()
const isProMode = ref(false)
let switchViewEvent = null
const hasLegacyProModeRuntime = () => {
  const legacyToggle = window.__legacyToggleProMode || window.toggleProMode
  return typeof legacyToggle === 'function'
}
const readProModeStateFromDom = () => {
  const overlay = document.getElementById('proModeOverlay')
  const bodyActive = document.body.classList.contains('pro-mode-active')
  const overlayActive = !!overlay?.classList.contains('active')
  const overlayVisible = !!overlay && overlay.style.display !== 'none'
  const toggle = document.getElementById('proModeToggle')
  const toggleActive = !!toggle?.classList.contains('active')
  return bodyActive || (overlayActive && overlayVisible) || toggleActive
}
const syncProModeStateSoon = () => {
  requestAnimationFrame(() => {
    isProMode.value = readProModeStateFromDom()
  })
  setTimeout(() => {
    isProMode.value = readProModeStateFromDom()
  }, 350)
  setTimeout(() => {
    isProMode.value = readProModeStateFromDom()
  }, 1650)
}
const handleToggleProMode = () => {
  const legacyToggle = window.__legacyToggleProMode || window.toggleProMode
  if (typeof legacyToggle === 'function') {
    legacyToggle()
    syncProModeStateSoon()
```

```
    return
  }
  isProMode.value = !isProMode.value
}
const syncGlobalProMode = () => {
  const active = !!isProMode.value
  window.__XCAGI_IS_PRO_MODE = active
  window.dispatchEvent(new CustomEvent('xcagi:pro-mode-changed', {
    detail: { isProMode: active }
  }))
}
let proModeObserver = null
let onToggleProModeEvent = null
onMounted(() => {
  const syncProModeFromDom = () => {
    isProMode.value = readProModeStateFromDom()
  }
  window.setProModeEnabled = (enabled) => {
    const shouldEnable = !!enabled
    const active = readProModeStateFromDom()
    if (shouldEnable === active) {
      isProMode.value = active
      return
    }
    if (hasLegacyProModeRuntime()) {
      const legacyToggle = window.__legacyToggleProMode || window.toggleProMode
      legacyToggle()
      syncProModeStateSoon()
    } else {
      isProMode.value = shouldEnable
    }
  }
  onToggleProModeEvent = () => {
    handleToggleProMode()
  }
  window.addEventListener('xcagi:toggle-pro-mode', onToggleProModeEvent)
  proModeObserver = new MutationObserver(() => {
    syncProModeFromDom()
  })
  proModeObserver.observe(document.body, { attributes: true, attributeFilter: ['class'] })
  const overlay = document.getElementById('proModeOverlay')
  if (overlay) {
    proModeObserver.observe(overlay, { attributes: true, attributeFilter: ['class', 'style'] })
  }
  syncProModeFromDom()
  syncGlobalProMode()
  const bindOnce = (id, eventName, handler) => {
    const el = document.getElementById(id)
    if (!el) return
    if (el.getAttribute('data-xcagi-bound') === '1') return
```

```
    el.setAttribute('data-xcagi-bound', '1')
    el.addEventListener(eventName, handler)
  }
  bindOnce('fileUploadEntry', 'click', () => {
    try {
      if (typeof window.openImportWindow === 'function') {
        console.log('[xcagi] click fileUploadEntry -> openImportWindow()')
        window.openImportWindow()
      } else {
        console.warn('[xcagi] openImportWindow not found on window')
      }
    } catch (err) {
      console.warn('[xcagi] fileUploadEntry click failed:', err)
    }
  })
  bindOnce('chooseFileBtn', 'click', () => {
    const fileInput = document.getElementById('fileInput')
    if (fileInput) fileInput.click()
  })
  switchViewEvent = (event) => {
    const view = event.detail?.view
    if (view) {
      console.log('[App] xcagi:switch-view received, navigating to:', view)
      router.push({ name: view })
    }
  }
  window.addEventListener('xcagi:switch-view', switchViewEvent)
})
watch(isProMode, () => {
  syncGlobalProMode()
})
onBeforeUnmount(() => {
  if (proModeObserver) {
    proModeObserver.disconnect()
    proModeObserver = null
  }
  if (onToggleProModeEvent) {
    window.removeEventListener('xcagi:toggle-pro-mode', onToggleProModeEvent)
    onToggleProModeEvent = null
  }
  if (switchViewEvent) {
    window.removeEventListener('xcagi:switch-view', switchViewEvent)
    switchViewEvent = null
  }
  if (window.setProModeEnabled) {
    delete window.setProModeEnabled
  }
  if (typeof window.__XCAGI_IS_PRO_MODE !== 'undefined') {
    delete window.__XCAGI_IS_PRO_MODE
  }
```

```
})
</script>
<template>
 <div class="transition-overlay" id="transitionOverlay"></div>
 <div class="preview-float-window" id="previewFloatWindow">
  <div class="preview-header">
   <h4><i class="fa fa-television" aria-hidden="true"></i> 媒体预览</h4>
   <button class="preview-close" id="previewCloseBtn" data-close-action="closePreviewWindow">&times;</button>
  </div>
  <div class="preview-content">
   <div class="preview-media" id="previewMedia">
    <div class="preview-placeholder">暂无预览内容</div>
   </div>
  </div>
 </div>
 <div class="progress-panel" id="progressPanel">
  <div class="progress-panel-header">
   <div class="progress-title">任务进度</div>
   <button class="progress-close" id="progressCloseBtn" data-close-action="hideProgress">&times;</button>
  </div>
  <div class="progress-info">
   <span class="progress-status">处理中...</span>
   <span class="progress-percent" id="progressPercent">0%</span>
  </div>
  <div class="progress-bar-wrapper">
   <div class="progress-bar-fill" id="progressBarFill" style="width: 0%;"></div>
  </div>
  <div class="progress-task" id="progressTask">正在初始化任务...</div>
 </div>
 <div class="file-upload-entry" id="fileUploadEntry">
  <span class="entry-icon" aria-hidden="true"><i class="fa fa-folder-open-o"></i></span>
  <span class="entry-text">上传文件</span>
 </div>
 <div class="import-float-window" id="importFloatWindow">
  <div class="import-header">
   <h4><i class="fa fa-folder-open-o" aria-hidden="true"></i> 导入文件</h4>
   <button class="import-close" id="importCloseBtn" data-close-action="closeImportWindow">&times;</button>
  </div>
  <div class="import-content">
   <div class="drop-zone" id="dropZone">
    <div class="drop-zone-icon" aria-hidden="true"><i class="fa fa-folder-open"></i></div>
    <div class="drop-zone-text">拖拽文件到此处或点击选择</div>
    <div class="drop-zone-hint">支持 Excel、CSV、图片等文件</div>
   </div>
   <input type="file" class="file-input" id="fileInput" multiple accept="*/*">
   <div class="import-actions">
    <button class="btn btn-primary" id="chooseFileBtn">选择文件</button>
    <button class="btn btn-success" id="openCameraBtn"><i class="fa fa-camera" aria-hidden="true"></i> 拍照识别</butto ...
    <button class="btn btn-secondary" id="cancelImportBtn" data-close-action="closeImportWindow">取消</button>
   </div>
```

```
    <div class="camera-panel" id="cameraPanel" style="display: none;">
     <video id="cameraVideo" autoplay playsinline></video>
     <canvas id="cameraCanvas" style="display: none;"></canvas>
     <div class="camera-buttons">
      <button class="btn btn-primary" id="capturePhotoBtn">拍照</button>
      <button class="btn btn-secondary" id="closeCameraBtn" data-close-action="closeCamera">关闭</button>
     </div>
    </div>
    <div class="import-progress" id="importProgress">
     <div class="progress-bar-container">
      <div class="progress-bar" id="progressBar"></div>
     </div>
     <div class="progress-text">
      <span id="progressText">读取中...</span>
      <span id="importProgressPercent">0%</span>
     </div>
    </div>
    <div class="import-status" id="importStatus"></div>
   </div>
  </div>
  <div class="import-float-window" id="labelsExportWindow">
   <div class="import-header">
    <h4><i class="fa fa-tag" aria-hidden="true"></i> 商标导出</h4>
    <button class="import-close" id="labelsExportCloseBtn" type="button" title="关闭">&times;</button>
   </div>
   <div class="import-content">
    <div id="labelsExportList" class="labels-export-list">
     <p class="labels-export-hint">加载中...</p>
    </div>
    <div class="import-actions" style="margin-top: 12px;">
     <button class="btn btn-secondary" id="labelsExportCloseBtn2" type="button">关闭</button>
    </div>
   </div>
  </div>
  <div class="import-float-window" id="printPanelWindow">
   <div class="import-header">
    <h4><i class="fa fa-print" aria-hidden="true"></i> 标签打印</h4>
    <button class="import-close" id="printPanelCloseBtn" type="button" title="关闭">&times;</button>
   </div>
   <div class="import-content">
    <div id="printPanelStatus" class="labels-export-hint">正在连接打印机...</div>
    <div id="printPanelProgress" class="print-panel-progress" style="display:none; margin:10px 0;"></div>
    <div id="printPanelResults" class="print-panel-results" style="margin:10px 0; max-height:240px; overflow-y:auto; ...
    <div class="import-actions" style="margin-top: 12px;">
     <button class="btn btn-primary" id="printPanelStartBtn" type="button">开始打印</button>
     <button class="btn btn-secondary" id="printPanelCloseBtn2" type="button">关闭</button>
    </div>
   </div>
  </div>
  <div id="labelFloatPreviews" class="label-float-previews hidden" aria-hidden="true"></div>
```

```
 <div id="labelPreviewModal" class="label-preview-modal hidden" aria-hidden="true">
  <div class="label-preview-modal-backdrop"></div>
  <div class="label-preview-modal-content">
   <img id="labelPreviewModalImg" src="" alt="标签预览" />
   <div class="label-preview-modal-actions">
    <a id="labelPreviewModalDownload" href="" download="" class="btn btn-primary">下载标签</a>
    <button type="button" class="btn btn-secondary label-preview-modal-close">关闭</button>
   </div>
  </div>
 </div>
 <ProMode v-model="isProMode" />
 <MainLayout
  :is-pro-mode="isProMode"
  @toggle-pro-mode="handleToggleProMode"
 >
  <router-view />
 </MainLayout>
</template>
<style>
</style>

// 文件：components\ConfirmDialog.vue
<template>
 <Teleport to="body">
  <div v-if="modelValue" class="confirm-dialog-overlay" @click.self="handleCancel">
   <div class="confirm-dialog" :style="{ maxWidth: maxWidth }">
    <div class="confirm-dialog-header">
     <h3>{{ title }}</h3>
    </div>
    <div class="confirm-dialog-body">
     <slot>
      <p>{{ message }}</p>
     </slot>
    </div>
    <div class="confirm-dialog-footer">
     <button
      v-if="showCancel"
      class="btn btn-secondary"
      @click="handleCancel"
     >
      {{ cancelText }}
     </button>
     <button
      :class="['btn', confirmClass]"
      @click="handleConfirm"
     >
      {{ confirmText }}
     </button>
    </div>
   </div>
```

```
    </div>
  </Teleport>
</template>
<script setup>
import { computed } from 'vue'
const props = defineProps({
  modelValue: {
    type: Boolean,
    default: false
  },
  title: {
    type: String,
    default: '确认操作'
  },
  message: {
    type: String,
    default: '确定要执行此操作吗？'
  },
  confirmText: {
    type: String,
    default: '确定'
  },
  cancelText: {
    type: String,
    default: '取消'
  },
  confirmClass: {
    type: String,
    default: 'btn-primary'
  },
  showCancel: {
    type: Boolean,
    default: true
  },
  maxWidth: {
    type: String,
    default: '400px'
  }
})
const emit = defineEmits([
  'update:modelValue',
  'confirm',
  'cancel'
])
const handleConfirm = () => {
  emit('confirm')
  emit('update:modelValue', false)
}
const handleCancel = () => {
  emit('cancel')
```

```
  emit('update:modelValue', false)
}
</script>
<style scoped>
.confirm-dialog-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 9999;
}
.confirm-dialog {
  background: white;
  border-radius: 8px;
  width: 90%;
  max-width: 400px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.confirm-dialog-header {
  padding: 16px 20px;
  border-bottom: 1px solid #e0e0e0;
}
.confirm-dialog-header h3 {
  margin: 0;
  font-size: 18px;
  font-weight: 600;
  color: #333;
}
.confirm-dialog-body {
  padding: 20px;
}
.confirm-dialog-body p {
  margin: 0;
  color: #666;
  line-height: 1.5;
}
.confirm-dialog-footer {
  padding: 16px 20px;
  border-top: 1px solid #e0e0e0;
  display: flex;
  gap: 10px;
  justify-content: flex-end;
}
.btn {
  padding: 8px 16px;
```

```
  border: none;
  border-radius: 4px;
  font-size: 14px;
  cursor: pointer;
  transition: background-color 0.2s;
}
.btn-primary {
  background-color: #3498db;
  color: white;
}
.btn-primary:hover {
  background-color: #2980b9;
}
.btn-secondary {
  background-color: #95a5a6;
  color: white;
}
.btn-secondary:hover {
  background-color: #7f8c8d;
}
.btn-danger {
  background-color: #e74c3c;
  color: white;
}
.btn-danger:hover {
  background-color: #c0392b;
}
.btn-success {
  background-color: #2ecc71;
  color: white;
}
.btn-success:hover {
  background-color: #27ae60;
}
</style>

// 文件：components\DataTable.vue
<template>
  <div class="data-table-wrapper" ref="tableWrapper" @scroll="handleScroll">
    <table class="data-table">
      <thead>
        <tr>
          <th v-if="selectable">
            <input type="checkbox" v-model="localSelectAll" @change.stop="handleSelectAll">
          </th>
          <th v-for="col in columns" :key="col.key">{{ col.label }}</th>
          <th v-if="$slots.actions">操作</th>
        </tr>
      </thead>
      <tbody>
```

```
      <tr v-if="loading && localData.length === 0">
        <td :colspan="totalColumns" class="empty-state">加载中...</td>
      </tr>
      <tr v-else-if="localData.length === 0 && !loading">
        <td :colspan="totalColumns" class="empty-state">{{ emptyText }}</td>
      </tr>
      <template v-else>
        <tr v-for="(row, index) in localData" :key="getRowKey(row, index)">
          <td v-if="selectable">
            <input
              type="checkbox"
              :value="getRowId(row)"
              v-model="localSelectedIds"
              @click.stop
              @change="handleCheckboxChange(row, $event)"
            >
          </td>
          <td v-for="col in columns" :key="col.key">
            <slot :name="`cell-${col.key}`" :row="row" :value="getCellValue(row, col)">
              {{ formatCell(row, col) }}
            </slot>
          </td>
          <td v-if="$slots.actions">
            <slot name="actions" :row="row" :index="index"></slot>
          </td>
        </tr>
      </template>
      <tr v-if="loading && localData.length > 0">
        <td :colspan="totalColumns" class="loading-state">加载中...</td>
      </tr>
      <tr v-if="hasMore && !loading && localData.length > 0">
        <td :colspan="totalColumns" class="has-more-tip">滚动加载更多</td>
      </tr>
      <tr v-if="!hasMore && localData.length > 0">
        <td :colspan="totalColumns" class="no-more-tip">没有更多数据了</td>
      </tr>
    </tbody>
  </table>
 </div>
</template>
<script setup>
import { ref, computed, watch, nextTick, useSlots } from 'vue'
const props = defineProps({
 columns: {
  type: Array,
  required: true
 },
 data: {
  type: Array,
  default: () => []
```

```
  },
  loading: {
    type: Boolean,
    default: false
  },
  selectable: {
    type: Boolean,
    default: false
  },
  selectedIds: {
    type: Array,
    default: () => []
  },
  rowKey: {
    type: String,
    default: 'id'
  },
  emptyText: {
    type: String,
    default: '暂无数据'
  },
  hasMore: {
    type: Boolean,
    default: true
  },
  height: {
    type: String,
    default: '500px'
  }
})
const emit = defineEmits([
  'update:selectedIds',
  'select-change',
  'row-click',
  'load-more'
])
const tableWrapper = ref(null)
const slots = useSlots()
const handleScroll = () => {
  if (!tableWrapper.value || props.loading || !props.hasMore) return
  const { scrollTop, scrollHeight, clientHeight } = tableWrapper.value
  const threshold = 100
  if (scrollHeight - scrollTop - clientHeight < threshold) {
    emit('load-more')
  }
}
watch(() => props.data, () => {
}, { deep: true })
const localSelectedIds = ref([...props.selectedIds])
const localSelectAll = ref(false)
```

```
let isSyncing = false
const totalColumns = computed(() => {
  let count = props.columns.length
  if (props.selectable) count++
  if (slots.actions) count++
  return count
})
const localData = computed(() => props.data)
watch(localSelectedIds, (newVal) => {
  if (isSyncing) return
  isSyncing = true
  emit('update:selectedIds', [...newVal])
  emit('select-change', [...newVal])
  nextTick(() => { isSyncing = false })
}, { deep: true })
watch(() => props.selectedIds, (newVal) => {
  if (isSyncing) return
  isSyncing = true
  localSelectedIds.value = [...newVal]
  nextTick(() => { isSyncing = false })
}, { deep: true })
const getRowId = (row) => {
  return row.id ?? row[props.rowKey] ?? row
}
const getRowKey = (row, index) => {
  return row[props.rowKey] ?? row.id ?? index
}
const getCellValue = (row, col) => {
  if (col.key.includes('.')) {
    return col.key.split('.').reduce((obj, key) => obj?.[key], row)
  }
  return row[col.key]
}
const formatCell = (row, col) => {
  const value = getCellValue(row, col)
  if (value === null || value === undefined) return col.default ?? '-'
  return value
}
const handleSelectAll = () => {
  if (localSelectAll.value) {
    localSelectedIds.value = localData.value.map((row, index) => getRowId(row))
  } else {
    localSelectedIds.value = []
  }
}
const handleCheckboxChange = (row, event) => {
  console.log('[DataTable] checkbox changed:', getRowId(row), event.target.checked)
}
const handleRowClick = (row, index) => {
  emit('row-click', { row, index })
```

```
}
defineExpose({
  clearSelection: () => {
    localSelectedIds.value = []
    localSelectAll.value = false
  },
  selectAll: () => {
    localSelectAll.value = true
    localSelectedIds.value = localData.value.map((row, index) => getRowId(row))
  }
})
</script>
<style scoped>
.data-table-wrapper {
  overflow-y: auto;
  overflow-x: auto;
  max-height: v-bind(height);
  border: 1px solid #e0e0e0;
  border-radius: 4px;
}
.data-table {
  width: 100%;
  border-collapse: collapse;
}
.data-table th,
.data-table td {
  padding: 10px 12px;
  text-align: left;
  border-bottom: 1px solid #e0e0e0;
}
.data-table th {
  background-color: #f8f9fa;
  font-weight: 600;
  color: #333;
  position: sticky;
  top: 0;
  z-index: 10;
}
.data-table tbody tr:hover {
  background-color: #f5f5f5;
}
.empty-state {
  text-align: center;
  color: #999;
  padding: 40px !important;
}
.loading-state,
.has-more-tip,
.no-more-tip {
  text-align: center;
```

```
  color: #999;
  font-size: 14px;
  padding: 12px !important;
}
.has-more-tip {
  color: #666;
}
.no-more-tip {
  color: #ccc;
}
</style>

// 文件：components\FileImport.vue
<template>
  <div class="file-import-overlay" v-if="modelValue" @click.self="handleClose">
    <div class="file-import-modal">
      <div class="file-import-header">
        <h4>{{ title }}</h4>
        <button class="close-btn" @click="handleClose" title="关闭">
          <i class="fas fa-times"></i>
        </button>
      </div>
      <div class="file-import-body">
        <div
          class="drop-zone"
          :class="{
            'dragover': isDragOver,
            'uploading': uploading
          }"
          @dragenter.prevent="handleDragEnter"
          @dragover.prevent="handleDragOver"
          @dragleave.prevent="handleDragLeave"
          @drop.prevent="handleDrop"
          @click="triggerFileInput"
        >
          <div class="drop-zone-content">
            <i class="fas fa-cloud-upload-alt"></i>
            <p class="drop-zone-title">点击或拖拽文件到此处</p>
            <p class="drop-zone-hint">{{ hint }}</p>
            <p class="drop-zone-supported">
              支持：Excel、CSV、图片、PDF、Word
            </p>
          </div>
          <input
            ref="fileInputRef"
            type="file"
            class="file-input"
            :accept="acceptFormats"
            :multiple="multiple"
            @change="handleFileChange"
```

```
      />
    </div>
    <div class="import-progress" v-if="uploading || progress > 0">
      <div class="progress-header">
        <span class="progress-text">{{ progressText }}</span>
        <span class="progress-percent">{{ progress }}%</span>
      </div>
      <div class="progress-bar">
        <div
          class="progress-fill"
          :style="{ width: progress + '%' }"
          :class="{ 'progress-animated': uploading }"
        ></div>
      </div>
    </div>
    <div class="import-status" v-if="status.show" :class="status.type">
      <i :class="statusIcon"></i>
      <span>{{ status.message }}</span>
    </div>
    <div class="file-list" v-if="selectedFiles.length > 0">
      <div class="file-list-header">
        <span>已选择文件 ({{ selectedFiles.length }})</span>
        <button class="clear-btn" @click="clearFiles" v-if="!uploading">
          <i class="fas fa-trash"></i> 清空
        </button>
      </div>
      <div class="file-items">
        <div
          class="file-item"
          v-for="(file, index) in selectedFiles"
          :key="index"
        >
          <i :class="getFileIcon(file.type)" class="file-icon"></i>
          <div class="file-info">
            <span class="file-name">{{ file.name }}</span>
            <span class="file-size">{{ formatFileSize(file.size) }}</span>
          </div>
          <span class="file-type-tag">{{ file.fileType }}</span>
        </div>
      </div>
    </div>
  </div>
  <div class="file-import-footer">
    <button
      class="btn btn-secondary"
      @click="handleClose"
      :disabled="uploading"
    >
      取消
    </button>
```

```
      <button
        class="btn btn-primary"
        @click="handleUpload"
        :disabled="selectedFiles.length === 0 || uploading"
      >
        <i class="fas fa-upload" v-if="!uploading"></i>
        <i class="fas fa-spinner fa-spin" v-else></i>
        {{ uploading ? '上传中...' : '开始导入' }}
      </button>
    </div>
  </div>
 </div>
</template>
<script setup>
import { ref, computed, watch } from 'vue';
import useFileImport, { FILE_EXTENSIONS } from '../composables/useFileImport';
const props = defineProps({
 modelValue: {
   type: Boolean,
   default: false
 },
 title: {
   type: String,
   default: '文件导入'
 },
 purpose: {
   type: String,
   default: 'general',
   validator: (value) => ['general', 'product_import', 'customers_import', 'order_parse', 'materials_import'].include ...
 },
 hint: {
   type: String,
   default: '支持 Excel、CSV、图片等格式，自动识别并分析'
 },
 acceptFormats: {
   type: String,
   default: '.xlsx,.xls,.csv,.jpg,.jpeg,.png,.gif,.webp,.pdf,.doc,.docx'
 },
 multiple: {
   type: Boolean,
   default: true
 },
 autoUpload: {
   type: Boolean,
   default: false
 }
});
const emit = defineEmits(['update:modelValue', 'uploaded', 'error', 'progress', 'file-complete']);
const fileInputRef = ref(null);
const isDragOver = ref(false);
```

```
const selectedFiles = ref([]);
const {
  uploading,
  progress,
  progressText,
  status,
  detectFileType,
  resetState,
  uploadFile,
  uploadMultipleFiles
} = useFileImport();
const computedHint = computed(() => {
  if (props.purpose === 'customers_import') {
    return '请上传购买单位列表 .xlsx（需含「单位名称」列），将校验格式并更新联系人/电话/地址';
  }
  if (props.purpose === 'product_import') {
    return '产品导入模式：支持任意文件，自动识别并分析';
  }
  if (props.purpose === 'order_parse') {
    return '订单解析模式：上传后自动提取订单关键信息';
  }
  if (props.purpose === 'materials_import') {
    return '原材料导入模式：支持 Excel/CSV 格式';
  }
  return props.hint;
});
const statusIcon = computed(() => {
  if (status.type === 'success') return 'fas fa-check-circle';
  if (status.type === 'error') return 'fas fa-exclamation-circle';
  return 'fas fa-info-circle';
});
function getFileIcon(fileType) {
  const icons = {
    excel: 'fas fa-file-excel',
    csv: 'fas fa-file-csv',
    image: 'fas fa-file-image',
    pdf: 'fas fa-file-pdf',
    word: 'fas fa-file-word',
    other: 'fas fa-file'
  };
  return icons[fileType] || icons.other;
}
function formatFileSize(bytes) {
  if (bytes === 0) return '0 B';
  const k = 1024;
  const sizes = ['B', 'KB', 'MB', 'GB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return Math.round(bytes / Math.pow(k, i) * 100) / 100 + ' ' + sizes[i];
}
function triggerFileInput() {
```

```
  if (!uploading.value && fileInputRef.value) {
    fileInputRef.value.click();
  }
}
function handleDragEnter(e) {
  if (!uploading.value) {
    isDragOver.value = true;
  }
}
function handleDragOver(e) {
  if (!uploading.value) {
    isDragOver.value = true;
  }
}
function handleDragLeave(e) {
  if (e.relatedTarget === null || !e.currentTarget.contains(e.relatedTarget)) {
    isDragOver.value = false;
  }
}
function handleDrop(e) {
  isDragOver.value = false;
  if (uploading.value) return;
  const files = e.dataTransfer.files;
  if (files.length > 0) {
    handleFiles(files);
  }
}
function handleFileChange(e) {
  const files = e.target.files;
  if (files.length > 0) {
    handleFiles(files);
  }
}
function handleFiles(fileList) {
  const newFiles = [];
  for (let i = 0; i < fileList.length; i++) {
    const file = fileList[i];
    const fileType = detectFileType(file);
    newFiles.push({
      ...file,
      fileType
    });
  }
  if (props.multiple) {
    selectedFiles.value = [...selectedFiles.value, ...newFiles];
  } else {
    selectedFiles.value = [newFiles[0]];
  }
  if (props.autoUpload && newFiles.length > 0) {
    handleUpload();
```

```
 }
}
function clearFiles() {
 selectedFiles.value = [];
 resetState();
}
async function handleUpload() {
 if (selectedFiles.value.length === 0 || uploading.value) return;
 const filesToUpload = selectedFiles.value.map(f => {
  const newFile = new File([f], f.name, { type: f.type });
  return newFile;
 });
 try {
  if (filesToUpload.length === 1) {
   const result = await uploadFile(filesToUpload[0], props.purpose, (percent, fileName) => {
    emit('progress', { percent, fileName, totalFiles: 1 });
   });
   if (result) {
    emit('uploaded', {
     file: selectedFiles.value[0],
     result
    });
   } else {
    emit('error', {
     file: selectedFiles.value[0],
     error: status.message
    });
   }
  } else {
   const results = await uploadMultipleFiles(filesToUpload, props.purpose, (fileResult, completed, total) => {
    emit('file-complete', {
     file: fileResult,
     completed,
     total
    });
   });
   emit('uploaded', {
    files: selectedFiles.value,
    results
   });
  }
 } catch (err) {
  console.error('Upload error:', err);
  emit('error', {
   error: err.message
  });
 }
}
function handleClose() {
 if (!uploading.value) {
```

```
    emit('update:modelValue', false);
    selectedFiles.value = [];
    resetState();
  }
}
watch(() => props.modelValue, (newVal) => {
  if (!newVal) {
    selectedFiles.value = [];
    resetState();
  }
});
</script>
<style scoped>
.file-import-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 9999;
  animation: fadeIn 0.3s ease;
}
@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}
.file-import-modal {
  background: white;
  border-radius: 12px;
  width: 90%;
  max-width: 600px;
  max-height: 80vh;
  display: flex;
  flex-direction: column;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
  animation: slideIn 0.3s ease;
}
@keyframes slideIn {
  from {
    transform: translateY(-20px);
    opacity: 0;
  }
```

```
  to {
    transform: translateY(0);
    opacity: 1;
  }
}
.file-import-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 24px;
  border-bottom: 1px solid #e0e0e0;
}
.file-import-header h4 {
  margin: 0;
  font-size: 18px;
  font-weight: 600;
  color: #333;
}
.close-btn {
  background: none;
  border: none;
  font-size: 20px;
  color: #999;
  cursor: pointer;
  padding: 4px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 4px;
  transition: all 0.2s;
}
.close-btn:hover {
  background: #f5f5f5;
  color: #333;
}
.file-import-body {
  padding: 24px;
  overflow-y: auto;
  flex: 1;
}
.drop-zone {
  border: 2px dashed #ccc;
  border-radius: 8px;
  padding: 40px 20px;
  text-align: center;
  cursor: pointer;
  transition: all 0.3s ease;
  background: #fafafa;
}
.drop-zone:hover {
```

```
  border-color: #667eea;
  background: #f0f4ff;
}
.drop-zone.dragover {
  border-color: #667eea;
  background: #e8eeff;
  transform: scale(1.02);
}
.drop-zone.uploading {
  cursor: not-allowed;
  opacity: 0.7;
}
.drop-zone-content {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 12px;
}
.drop-zone-content i {
  font-size: 48px;
  color: #667eea;
  margin-bottom: 8px;
}
.drop-zone-title {
  font-size: 16px;
  font-weight: 600;
  color: #333;
  margin: 0;
}
.drop-zone-hint {
  font-size: 13px;
  color: #666;
  margin: 0;
}
.drop-zone-supported {
  font-size: 12px;
  color: #999;
  margin: 0;
}
.file-input {
  display: none;
}
.import-progress {
  margin-top: 20px;
}
.progress-header {
  display: flex;
  justify-content: space-between;
  margin-bottom: 8px;
  font-size: 13px;
```

```
  color: #666;
}
.progress-percent {
  font-weight: 600;
  color: #667eea;
}
.progress-bar {
  height: 8px;
  background: #e0e0e0;
  border-radius: 4px;
  overflow: hidden;
}
.progress-fill {
  height: 100%;
  background: linear-gradient(90deg, #667eea, #764ba2);
  transition: width 0.3s ease;
}
.progress-fill.progress-animated {
  animation: progressPulse 1.5s ease-in-out infinite;
}
@keyframes progressPulse {
  0%, 100% {
    opacity: 1;
  }
  50% {
    opacity: 0.7;
  }
}
.import-status {
  margin-top: 16px;
  padding: 12px 16px;
  border-radius: 6px;
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 14px;
  animation: slideDown 0.3s ease;
}
@keyframes slideDown {
  from {
    opacity: 0;
    transform: translateY(-10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
.import-status.success {
  background: #d4edda;
```

```
  color: #155724;
  border: 1px solid #c3e6cb;
}
.import-status.error {
  background: #f8d7da;
  color: #721c24;
  border: 1px solid #f5c6cb;
}
.file-list {
  margin-top: 20px;
  border: 1px solid #e0e0e0;
  border-radius: 6px;
  overflow: hidden;
}
.file-list-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 16px;
  background: #f9f9f9;
  border-bottom: 1px solid #e0e0e0;
  font-size: 14px;
  font-weight: 600;
  color: #333;
}
.clear-btn {
  background: none;
  border: none;
  color: #dc3545;
  cursor: pointer;
  font-size: 13px;
  display: flex;
  align-items: center;
  gap: 4px;
  padding: 4px 8px;
  border-radius: 4px;
  transition: all 0.2s;
}
.clear-btn:hover {
  background: #fee;
}
.file-items {
  max-height: 200px;
  overflow-y: auto;
}
.file-item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 16px;
```

```
  border-bottom: 1px solid #f0f0f0;
  transition: background 0.2s;
}
.file-item:last-child {
  border-bottom: none;
}
.file-item:hover {
  background: #f9f9f9;
}
.file-icon {
  font-size: 24px;
  color: #667eea;
}
.file-info {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 4px;
}
.file-name {
  font-size: 14px;
  color: #333;
  font-weight: 500;
}
.file-size {
  font-size: 12px;
  color: #999;
}
.file-type-tag {
  font-size: 11px;
  padding: 2px 8px;
  background: #e8eeff;
  color: #667eea;
  border-radius: 4px;
  text-transform: uppercase;
}
.file-import-footer {
  display: flex;
  justify-content: flex-end;
  gap: 12px;
  padding: 16px 24px;
  border-top: 1px solid #e0e0e0;
}
.btn {
  padding: 10px 20px;
  border-radius: 6px;
  font-size: 14px;
  font-weight: 500;
  cursor: pointer;
  display: flex;
```

```
  align-items: center;
  gap: 8px;
  transition: all 0.2s;
  border: none;
}
.btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
.btn-secondary {
  background: #f5f5f5;
  color: #333;
}
.btn-secondary:hover:not(:disabled) {
  background: #e0e0e0;
}
.btn-primary {
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: white;
}
.btn-primary:hover:not(:disabled) {
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(102, 126, 234, 0.4);
}
</style>

// 文件：components\HelloWorld.vue
<script setup>
import { ref } from 'vue'
defineProps({
  msg: String,
})
const count = ref(0)
</script>
<template>
  <h1>{{ msg }}</h1>
  <div class="card">
    <button type="button" @click="count++">count is {{ count }}</button>
    <p>
      Edit
      <code>components/HelloWorld.vue</code> to test HMR
    </p>
  </div>
  <p>
    Check out
    <a href="https:
      >create-vue</a
    >, the official Vue + Vite starter
  </p>
  <p>
```

```
   Install
   <a href="https:
   in your IDE for a better DX
 </p>
 <p class="read-the-docs">Click on the Vite and Vue logos to learn more</p>
</template>
<style scoped>
.read-the-docs {
 color: #888;
}
</style>

// 文件：components\IndustrySelector.vue
<template>
 <div class="industry-selector" :class="{ 'is-expanded': isExpanded }">
  <div class="selector-trigger" @click="toggleExpand">
   <span class="industry-icon"><i class="fa fa-industry" aria-hidden="true"></i></span>
   <span class="industry-name">{{ currentIndustryName }}</span>
   <span class="industry-unit">{{ primaryUnit }}</span>
   <span class="expand-icon" :class="{ rotated: isExpanded }">▼</span>
  </div>
  <div v-if="isExpanded" class="selector-dropdown">
   <div class="dropdown-list">
    <div
     v-for="industry in industries"
     :key="industry.id"
     class="dropdown-item"
     :class="{ active: industry.id === currentIndustryId }"
     @click="selectIndustry(industry.id)"
    >
     <span class="item-name">{{ industry.name }}</span>
     <span class="item-unit">{{ getIndustryUnit(industry.id) }}</span>
     <span class="item-check" v-if="industry.id === currentIndustryId"><i class="fa fa-check" aria-hidden="true"> ...
    </div>
   </div>
  </div>
  <div v-if="isExpanded" class="dropdown-overlay" @click="closeDropdown"></div>
 </div>
</template>
<script setup>
import { ref, computed, onMounted } from 'vue'
import { useIndustryStore } from '@/stores/industry'
const industryStore = useIndustryStore()
const isExpanded = ref(false)
const industries = computed(() => industryStore.industries)
const currentIndustryId = computed(() => industryStore.currentIndustryId)
const currentIndustryName = computed(() => industryStore.currentIndustry?.name || '加载中...')
const primaryUnit = computed(() => industryStore.currentConfig?.units?.primary || '')
const getIndustryUnit = (industryId) => {
 const industry = industries.value.find(i => i.id === industryId)
```

```
  return ''
}
const toggleExpand = () => {
  isExpanded.value = !isExpanded.value
}
const closeDropdown = () => {
  isExpanded.value = false
}
const selectIndustry = async (industryId) => {
  if (industryId === currentIndustryId.value) {
    closeDropdown()
    return
  }
  const success = await industryStore.switchIndustry(industryId)
  if (success) {
    closeDropdown()
  }
}
onMounted(async () => {
  if (!industryStore.isLoaded) {
    await industryStore.initialize()
  }
})
</script>
<style scoped>
.industry-selector {
  position: relative;
}
.selector-trigger {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 10px 15px;
  cursor: pointer;
  transition: background 0.15s ease;
  user-select: none;
  color: rgba(255, 255, 255, 0.85);
  border-radius: 4px;
  margin: 2px 8px;
}
.selector-trigger:hover {
  background: rgba(255, 255, 255, 0.1);
}
.industry-icon {
  font-size: 14px;
}
.industry-name {
  font-weight: 500;
  flex: 1;
}
```

```
.industry-unit {
  font-size: 12px;
  padding: 2px 6px;
  background: rgba(79, 172, 254, 0.2);
  color: #4facfe;
  border-radius: 4px;
}
.expand-icon {
  font-size: 10px;
  color: rgba(255, 255, 255, 0.5);
  transition: transform 0.2s ease;
}
.expand-icon.rotated {
  transform: rotate(180deg);
}
.selector-dropdown {
  position: absolute;
  top: 100%;
  left: 8px;
  right: 8px;
  background: #1a1a2e;
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 6px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.3);
  z-index: 1000;
  overflow: hidden;
  margin-top: 4px;
}
.dropdown-list {
  max-height: 250px;
  overflow-y: auto;
}
.dropdown-item {
  display: flex;
  align-items: center;
  padding: 10px 12px;
  cursor: pointer;
  transition: background 0.15s ease;
  color: rgba(255, 255, 255, 0.85);
}
.dropdown-item:hover {
  background: rgba(255, 255, 255, 0.05);
}
.dropdown-item.active {
  background: rgba(79, 172, 254, 0.15);
  color: #fff;
}
.item-name {
  flex: 1;
}
```

```
.item-unit {
  font-size: 11px;
  padding: 2px 5px;
  background: rgba(79, 172, 254, 0.15);
  color: #4facfe;
  border-radius: 3px;
  margin-right: 8px;
}
.item-check {
  color: #4facfe;
  font-weight: bold;
  margin-left: 8px;
}
.dropdown-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  z-index: 999;
}
</style>

// 文件：components\MainLayout.vue
<template>
  <div class="main-container">
    <Sidebar
      :active-view="currentRouteName"
      :is-pro-mode="isProMode"
      @change-view="handleViewChange"
      @toggle-pro-mode="$emit('toggle-pro-mode')"
    />
    <div class="main-content">
      <div class="top-bar">
        <div class="page-title">{{ currentViewTitle }}</div>
        <div class="mode-badge" :class="props.isProMode ? 'pro' : 'normal'">
          {{ props.isProMode ? '专业版' : '普通版' }}
        </div>
        <TopAssistantFloat />
      </div>
      <slot></slot>
    </div>
  </div>
</template>
<script setup>
import { computed, onMounted } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { useIndustryStore } from '@/stores/industry'
import Sidebar from './Sidebar.vue'
import TopAssistantFloat from './TopAssistantFloat.vue'
```

```
ai_request_errors_total = Counter(
  'ai_request_errors_total',
  'Total number of AI request errors',
  ['service', 'error_type']
)
active_requests = Gauge(
  'active_requests',
  'Number of active requests'
)
circuit_breaker_state = Gauge(
  'circuit_breaker_state',
  'Circuit breaker state (0=closed, 1=half_open, 2=open)',
  ['name']
)
circuit_breaker_failures_total = Counter(
  'circuit_breaker_failures_total',
  'Total number of circuit breaker failures',
  ['name', 'circuit_state']
)
app_info = Info(
  'app',
  'Application information'
)
def init_metrics(app_name: str, version: str):
  app_info.info({
    'name': app_name,
    'version': version
  })
def metrics_endpoint() -> Response:
  return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)
def track_request_duration(method: str, endpoint: str):
  def decorator(func: Callable) -> Callable:
    @wraps(func)
    def wrapper(*args, **kwargs) -> Any:
      active_requests.inc()
      start_time = time.time()
      try:
        result = func(*args, **kwargs)
        duration = time.time() - start_time
        api_request_duration_seconds.labels(
          method=method,
          endpoint=endpoint
        ).observe(duration)
        return result
      finally:
        active_requests.dec()
    return wrapper
  return decorator
def track_ai_request(service: str):
  def decorator(func: Callable) -> Callable:
```

```
        @wraps(func)
        def wrapper(*args, **kwargs) -> Any:
            start_time = time.time()
            try:
                result = func(*args, **kwargs)
                duration = time.time() - start_time
                ai_requests_total.labels(service=service, status='success').inc()
                ai_request_duration_seconds.labels(service=service).observe(duration)
                return result
            except Exception as e:
                ai_requests_total.labels(service=service, status='error').inc()
                ai_request_errors_total.labels(service=service, error_type=type(e).__name__).inc()
                raise
        return wrapper
    return decorator

// 文件：utils\path_utils.py
import os
import sys
def get_base_dir() -> str:
    if hasattr(sys, '_MEIPASS'):
        return sys._MEIPASS
    return os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
def get_resources_dir() -> str:
    return os.path.join(get_base_dir(), "resources")
def get_resource_path(*parts: str) -> str:
    return os.path.join(get_resources_dir(), *parts)
def get_app_data_dir() -> str:
    base = get_base_dir()
    if hasattr(sys, '_MEIPASS'):
        app_data = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
        app_data_dir = os.path.join(app_data, 'XCAGI')
    else:
        app_data_dir = base
    os.makedirs(app_data_dir, exist_ok=True)
    return app_data_dir
def get_data_dir() -> str:
    return os.path.join(get_app_data_dir(), 'data')
def get_upload_dir() -> str:
    upload_dir = os.path.join(get_app_data_dir(), 'uploads')
    os.makedirs(upload_dir, exist_ok=True)
    return upload_dir
def get_log_dir() -> str:
    log_dir = os.path.join(get_app_data_dir(), 'logs')
    os.makedirs(log_dir, exist_ok=True)
    return log_dir
def get_db_path(db_name: str = 'products.db') -> str:
    return os.path.join(get_data_dir(), db_name)
def ensure_dir(directory: str) -> str:
    os.makedirs(directory, exist_ok=True)
```

```
    return directory

// 文件：utils\printer_automation.py
import logging
import subprocess
import time
from typing import Dict, List, Optional, Tuple
import win32api
import win32con
import win32gui
import win32print
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
class PrinterAutomation:
    def __init__(self):
        self.current_printer = None
        self.original_default = None
    def move_mouse(self, x: int, y: int):
        win32api.SetCursorPos((x, y))
        time.sleep(0.2)
    def click_mouse(self, x: int, y: int, button: str = 'left'):
        self.move_mouse(x, y)
        if button == 'left':
            win32api.mouse_event(win32con.MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0)
            time.sleep(0.1)
            win32api.mouse_event(win32con.MOUSEEVENTF_LEFTUP, 0, 0, 0, 0)
        elif button == 'right':
            win32api.mouse_event(win32con.MOUSEEVENTF_RIGHTDOWN, 0, 0, 0, 0)
            time.sleep(0.1)
            win32api.mouse_event(win32con.MOUSEEVENTF_RIGHTUP, 0, 0, 0, 0)
        time.sleep(0.3)
    def find_window(self, title: str) -> int:
        def callback(hwnd, hwnds):
            if win32gui.IsWindowVisible(hwnd):
                window_title = win32gui.GetWindowText(hwnd)
                if title in window_title:
                    hwnds.append(hwnd)
            return True
        hwnds = []
        win32gui.EnumWindows(callback, hwnds)
        return hwnds[0] if hwnds else 0
    def get_window_position(self, hwnd: int) -> Tuple[int, int, int, int]:
        rect = win32gui.GetWindowRect(hwnd)
        return rect
    def set_default_printer(self, printer_name: str) -> bool:
        try:
            logger.info(f"设置Windows默认打印机为: {printer_name}")
            result = subprocess.run([
                'rundll32', 'printui.dll,PrintUIEntry',
                '/y', '/n', printer_name
```

```
        ], capture_output=True, text=True, timeout=10)
        if result.returncode == 0:
            logger.info(f"成功设置默认打印机为: {printer_name}")
            time.sleep(1)
            return True
        else:
            logger.error(f"设置默认打印机失败: {result.stderr}")
            return False
    except Exception as e:
        logger.error(f"设置默认打印机异常: {e}")
        return False
def handle_printer_dialog(self, target_printer: str, timeout: int = 10) -> bool:
    start_time = time.time()
    while (time.time() - start_time) < timeout:
        hwnd = self.find_window('打印')
        if hwnd:
            logger.info(f"找到打印机对话框，窗口句柄: {hwnd}")
            rect = self.get_window_position(hwnd)
            logger.info(f"对话框位置: {rect}")
            center_x = (rect[0] + rect[2]) // 2
            center_y = (rect[1] + rect[3]) // 2
            printer_dropdown_x = center_x
            printer_dropdown_y = center_y - 100
            self.click_mouse(printer_dropdown_x, printer_dropdown_y)
            time.sleep(0.5)
            logger.info(f"尝试选择打印机: {target_printer}")
            ok_button_x = center_x + 100
            ok_button_y = center_y + 50
            self.click_mouse(ok_button_x, ok_button_y)
            logger.info("打印机对话框处理完成")
            return True
        time.sleep(0.5)
    logger.info("未找到打印机对话框")
    return False
def print_with_automation(self, file_path: str, target_printer: str) -> Dict:
    try:
        logger.info(f"开始自动化打印: {file_path} 到 {target_printer}")
        self.original_default = win32print.GetDefaultPrinter()
        logger.info(f"原始默认打印机: {self.original_default}")
        if self.original_default != target_printer:
            logger.info(f"临时设置默认打印机为: {target_printer}")
            self.set_default_printer(target_printer)
        logger.info("使用ShellExecute打印文件")
        result = win32api.ShellExecute(
            0,
            "print",
            file_path,
            f'/d:"{target_printer}"',
            ".",
            1
```

```
        )
        if result <= 32:
            raise Exception(f"ShellExecute失败，错误代码: {result}")
        logger.info(f"ShellExecute调用成功，结果: {result}")
        logger.info("检查是否需要处理打印机对话框...")
        self.handle_printer_dialog(target_printer)
        logger.info("等待打印完成...")
        time.sleep(5)
        if self.original_default and self.original_default != target_printer:
            logger.info(f"恢复原始默认打印机: {self.original_default}")
            self.set_default_printer(self.original_default)
        return {
            "success": True,
            "message": f"自动化打印成功发送到 {target_printer}",
            "printer": target_printer,
            "file": file_path
        }
    except Exception as e:
        logger.error(f"自动化打印失败: {e}")
        if self.original_default and self.current_printer != self.original_default:
            try:
                self.set_default_printer(self.original_default)
            except:
                pass
        return {
            "success": False,
            "message": f"自动化打印失败: {str(e)}"
        }
class EnhancedPrinterUtils:
    def __init__(self):
        self.automation = PrinterAutomation()
    def print_file_enhanced(self, file_path: str, printer_name: str, use_automation: bool = True, use_default_printer: ...
        try:
            if not use_automation:
                from app.utils.print_utils import PrinterUtils
                utils = PrinterUtils()
                return utils.print_file(file_path, printer_name, use_default_printer=use_default_printer)
            else:
                return self.automation.print_with_automation(file_path, printer_name)
        except Exception as e:
            logger.error(f"增强打印失败: {e}")
            return {
                "success": False,
                "message": f"增强打印失败: {str(e)}"
            }

// 文件：utils\print_utils.py
import logging
import os
import time
```

```
from typing import Dict, List, Optional
import pythoncom
import win32api
import win32print
logging.basicConfig(level=logging.INFO, encoding='utf-8')
logger = logging.getLogger(__name__)
class PrinterUtils:
    def __init__(self):
        self._com_initialized = False
    def _ensure_com_initialized(self):
        if not self._com_initialized:
            try:
                pythoncom.CoInitialize()
                self._com_initialized = True
            except Exception as e:
                logger.warning(f"COM初始化警告: {e}")
    def get_available_printers(self) -> List[Dict[str, str]]:
        try:
            self._ensure_com_initialized()
            printers = []
            try:
                default_printer = win32print.GetDefaultPrinter()
                logger.info(f"默认打印机: {default_printer}")
            except:
                default_printer = None
                logger.warning("无法获取默认打印机")
            all_printers = win32print.EnumPrinters(win32print.PRINTER_ENUM_LOCAL | win32print.PRINTER_ENUM_CONNECTIONS ...
            logger.info(f"找到 {len(all_printers)} 个打印机")
            for printer_info in all_printers:
                logger.info(f"打印机信息: {printer_info}")
                logger.info(f"打印机信息长度: {len(printer_info)}")
                if len(printer_info) >= 3:
                    printer_name = printer_info[2]
                    status = 0
                    if len(printer_info) > 6:
                        status = printer_info[6]
                    elif len(printer_info) > 5:
                        try:
                            status = int(printer_info[5]) if printer_info[5] else 0
                        except:
                            pass
                    status_text = self._get_printer_status(status)
                is_default = (printer_name == default_printer)
                printers.append({
                    "name": printer_name,
                    "status": status_text,
                    "is_default": is_default
                })
                logger.info(f"  - {printer_name} (默认: {is_default}, 状态: {status_text})")
            return printers
```

```
        except Exception as e:
            logger.error(f"获取打印机列表失败: {e}", exc_info=True)
            return []
    def _get_printer_status(self, status_code: int) -> str:
        status_map = {
            win32print.PRINTER_STATUS_PAUSED: "已暂停",
            win32print.PRINTER_STATUS_ERROR: "错误",
            win32print.PRINTER_STATUS_PENDING_DELETION: "正在删除",
            win32print.PRINTER_STATUS_PAPER_JAM: "卡纸",
            win32print.PRINTER_STATUS_PAPER_OUT: "缺纸",
            win32print.PRINTER_STATUS_MANUAL_FEED: "等待手动送纸",
            win32print.PRINTER_STATUS_PRINTING: "打印中",
            win32print.PRINTER_STATUS_OUTPUT_BIN_FULL: "输出纸盒已满",
            win32print.PRINTER_STATUS_NOT_AVAILABLE: "不可用",
            win32print.PRINTER_STATUS_WAITING: "等待中",
            win32print.PRINTER_STATUS_PROCESSING: "处理中",
            win32print.PRINTER_STATUS_INITIALIZING: "初始化中",
            win32print.PRINTER_STATUS_WARMING_UP: "预热中",
            win32print.PRINTER_STATUS_TONER_LOW: "墨粉不足",
            win32print.PRINTER_STATUS_NO_TONER: "无墨粉",
            win32print.PRINTER_STATUS_PAGE_PUNT: "页面跳过",
            win32print.PRINTER_STATUS_USER_INTERVENTION: "需要用户干预",
            win32print.PRINTER_STATUS_OUT_OF_MEMORY: "内存不足",
            win32print.PRINTER_STATUS_DOOR_OPEN: "前门打开",
            win32print.PRINTER_STATUS_SERVER_UNKNOWN: "服务器未知",
            win32print.PRINTER_STATUS_POWER_SAVE: "节能模式",
        }
        return status_map.get(status_code, "就绪")
    def monitor_print_job(self, printer_name: str, timeout: int = 60) -> bool:
        try:
            logger.info(f"开始监控打印机 {printer_name} 的打印任务...")
            start_time = time.time()
            while time.time() - start_time < timeout:
                try:
                    hPrinter = win32print.OpenPrinter(printer_name)
                    try:
                        jobs = win32print.EnumJobs(hPrinter, 0, -1, 1)
                        if not jobs:
                            logger.info("打印机队列为空，打印任务已完成")
                            return True
                        else:
                            logger.info(f"打印机队列中有 {len(jobs)} 个任务，等待完成...")
                            for i, job in enumerate(jobs):
                                logger.info(f"  任务 {i+1}: {job}")
                    finally:
                        win32print.ClosePrinter(hPrinter)
                except Exception as e:
                    logger.warning(f"监控打印任务失败: {e}")
                time.sleep(1)
            logger.warning(f"监控打印任务超时（{timeout}秒）")
```

```
            return False
        except Exception as e:
            logger.error(f"监控打印任务时发生错误: {e}")
            return False
    def print_file(self, file_path: str, printer_name: Optional[str] = None, use_default_printer: bool = False) -> Dic ...
        try:
            if not os.path.exists(file_path):
                return {
                    "success": False,
                    "message": f"文件不存在: {file_path}"
                }
            _, ext = os.path.splitext(file_path)
            ext = ext.lower()
            if not printer_name:
                logger.error("未指定打印机名称，拒绝打印")
                return {
                    "success": False,
                    "message": "未指定打印机名称，无法打印"
                }
            logger.info(f"准备打印文件: {file_path}")
            logger.info(f"使用打印机: {printer_name}")
            original_default_printer = None
            if use_default_printer:
                try:
                    original_default_printer = win32print.GetDefaultPrinter()
                    logger.info(f"当前默认打印机: {original_default_printer}")
                    logger.info(f"目标打印机: {printer_name}")
                    if original_default_printer != printer_name:
                        logger.info(f"正在修改默认打印机...")
                        try:
                            win32print.SetDefaultPrinter(printer_name)
                            logger.info(f"SetDefaultPrinter 调用完成")
                        except Exception as e:
                            logger.error(f"SetDefaultPrinter 调用失败: {e}")
                            import traceback
                            logger.error(traceback.format_exc())
                        time.sleep(0.5)
                        try:
                            new_default = win32print.GetDefaultPrinter()
                            logger.info(f"验证 - 当前默认打印机: {new_default}")
                        except Exception as e:
                            logger.error(f"GetDefaultPrinter 调用失败: {e}")
                            new_default = original_default_printer
                        if new_default == printer_name:
                            logger.info(f"默认打印机修改成功: {new_default}")
                        else:
                            logger.error(f"默认打印机修改失败！当前: {new_default}, 目标: {printer_name}")
                            try:
                                win32print.SetDefaultPrinter(printer_name)
                                time.sleep(0.5)
```

```
                    new_default = win32print.GetDefaultPrinter()
                    logger.info(f"第二次修改后默认打印机: {new_default}")
                except Exception as e:
                    logger.error(f"第二次修改失败: {e}")
            else:
                logger.info("当前默认打印机已经是目标打印机，无需修改")
        except Exception as e:
            logger.error(f"修改默认打印机失败: {e}")
            import traceback
            logger.error(traceback.format_exc())
    else:
        logger.info("已明确指定打印机，不使用系统默认打印机")
    print_result = None
    try:
        if ext in ['.xlsx', '.xls']:
            print_result = self._print_excel(file_path, printer_name)
        elif ext == '.pdf':
            print_result = self._print_pdf(file_path, printer_name)
        else:
            print_result = self._print_default(file_path, printer_name)
        if print_result.get('success', False):
            logger.info("打印命令已发送，继续执行后续操作")
    finally:
        if use_default_printer and original_default_printer:
            try:
                if original_default_printer != printer_name:
                    logger.info(f"恢复默认打印机为: {original_default_printer}")
                    win32print.SetDefaultPrinter(original_default_printer)
                    logger.info("默认打印机恢复成功")
            except Exception as e:
                logger.warning(f"恢复默认打印机失败: {e}")
    return print_result
  except Exception as e:
    logger.error(f"打印文件失败: {e}")
    return {
      "success": False,
      "message": f"打印失败: {str(e)}"
    }
def _print_excel(self, file_path: str, printer_name: str) -> Dict:
  try:
    logger.info(f"开始打印Excel文件: {file_path}")
    logger.info(f"使用打印机: {printer_name}")
    try:
      logger.info("方法1: 使用os.startfile打印")
      os.startfile(file_path, "print")
      logger.info(f"os.startfile打印成功: {file_path}")
      return {
        "success": True,
        "message": "打印任务已发送（os.startfile）",
        "file": os.path.basename(file_path),
```

```
            "printer": printer_name
        }
    except Exception as e1:
        logger.warning(f"方法1失败: {e1}")
        try:
            logger.info("方法2: 使用ShellExecute print")
            result = win32api.ShellExecute(
                0,
                "print",
                file_path,
                None,
                ".",
                1
            )
            if result > 32:
                logger.info(f"ShellExecute打印成功: {file_path}")
                return {
                    "success": True,
                    "message": "打印任务已发送到打印机",
                    "file": os.path.basename(file_path),
                    "printer": printer_name
                }
            else:
                raise Exception(f"ShellExecute失败，错误代码: {result}")
        except Exception as e2:
            logger.warning(f"方法2失败: {e2}")
            try:
                logger.info("方法3: 打开文件让用户手动打印")
                os.startfile(file_path)
                logger.info(f"已打开文件: {file_path}")
                return {
                    "success": True,
                    "message": "文件已打开，请手动打印",
                    "file": os.path.basename(file_path),
                    "printer": printer_name,
                    "manual": True
                }
            except Exception as e3:
                logger.error(f"方法3也失败: {e3}")
                raise Exception(f"所有打印方法都失败: {e1}; {e2}; {e3}")
    except Exception as e:
        logger.error(f"打印Excel文件失败: {e}")
        return {
            "success": False,
            "message": f"打印失败: {str(e)}"
        }
def _print_pdf(self, file_path: str, printer_name: str) -> Dict:
    try:
        logger.info(f"尝试使用win32print直接打印PDF到 {printer_name}")
        try:
```

```
            hprinter = win32print.OpenPrinter(printer_name)
            try:
                with open(file_path, 'rb') as f:
                    file_data = f.read()
                job_id = win32print.StartDocPrinter(hprinter, 1, ("PDF Job", None, "RAW"))
                win32print.StartPagePrinter(hprinter)
                win32print.WritePrinter(hprinter, file_data)
                win32print.EndPagePrinter(hprinter)
                win32print.EndDocPrinter(hprinter)
                logger.info(f"PDF文件已通过win32print发送到打印机: {file_path}")
                return {
                    "success": True,
                    "message": "PDF文件已发送到打印机",
                    "file": os.path.basename(file_path),
                    "printer": printer_name,
                    "method": "win32print"
                }
            finally:
                win32print.ClosePrinter(hprinter)
        except Exception as e:
            logger.warning(f"win32print打印失败: {e}")
        logger.info("尝试使用subprocess调用外部程序")
        try:
            import subprocess
            adobe_paths = [
                r"C:\Program Files (x86)\Adobe\Acrobat Reader DC\Reader\AcroRd32.exe",
                r"C:\Program Files\Adobe\Acrobat Reader DC\Reader\AcroRd32.exe",
            ]
            for adobe_path in adobe_paths:
                if os.path.exists(adobe_path):
                    logger.info(f"使用Adobe Reader: {adobe_path}")
                    result = subprocess.run([
                        adobe_path,
                        "/t",
                        file_path,
                        printer_name
                    ], check=True, timeout=30, capture_output=True)
                    logger.info(f"PDF文件已通过Adobe Reader发送到打印机")
                    return {
                        "success": True,
                        "message": "PDF文件已通过Adobe Reader发送到打印机",
                        "file": os.path.basename(file_path),
                        "printer": printer_name,
                        "method": "adobe_cli"
                    }
            logger.warning("未找到Adobe Reader")
        except Exception as e:
            logger.warning(f"subprocess方法失败: {e}")
        raise Exception("所有PDF打印方法都失败")
    except Exception as e:
```

```
        logger.error(f"打印PDF文件失败: {e}")
        return {
            "success": False,
            "message": f"打印PDF失败: {str(e)}"
        }
    def _print_default(self, file_path: str, printer_name: str, show_app: bool = False) -> Dict:
        try:
            show_cmd = 1 if show_app else 0
            win32api.ShellExecute(
                0,
                "print",
                file_path,
                f'/d:"{printer_name}"',
                ".",
                show_cmd
            )
            app_status = "（显示应用窗口）" if show_app else "（隐藏应用窗口）"
            logger.info(f"文件已发送到打印机{app_status}: {file_path}")
            return {
                "success": True,
                "message": f"打印任务已发送到打印机{app_status}",
                "file": os.path.basename(file_path),
                "printer": printer_name,
                "show_app": show_app
            }
        except Exception as e:
            logger.error(f"打印文件失败: {e}")
            return {
                "success": False,
                "message": f"打印失败: {str(e)}"
            }
    def get_default_printer(self) -> Optional[str]:
        try:
            return win32print.GetDefaultPrinter()
        except Exception as e:
            logger.error(f"获取默认打印机失败: {e}")
            return None
    def test_printer(self, printer_name: str) -> Dict:
        try:
            hprinter = win32print.OpenPrinter(printer_name)
            printer_info = win32print.GetPrinter(hprinter, 2)
            status = printer_info['Status']
            status_text = self._get_printer_status(status)
            win32print.ClosePrinter(hprinter)
            return {
                "success": True,
                "available": True,
                "printer": printer_name,
                "status": status_text
            }
```

```
        except Exception as e:
            logger.error(f"测试打印机失败: {e}")
            return {
                "success": False,
                "available": False,
                "printer": printer_name,
                "message": str(e)
            }
    def get_document_printer(self) -> Optional[str]:
        try:
            printers = self.get_available_printers()
            if not printers:
                return None
            keywords = ['joli', '24-pin', 'dot matrix', 'impact', 'lq', '针式', 'hp', 'canon', 'epson']
            for printer in printers:
                name_lower = printer['name'].lower()
                if any(kw in name_lower for kw in keywords):
                    return printer['name']
            return printers[0]['name'] if printers else None
        except Exception as e:
            logger.error(f"获取发货单打印机失败: {e}")
            return None
    def get_label_printer(self) -> Optional[str]:
        try:
            printers = self.get_available_printers()
            if not printers:
                return None
            keywords = ['tsc', 'ttp', 'label', '标签', 'thermal', 'barcode', 'zebra']
            for printer in printers:
                name_lower = printer['name'].lower()
                if any(kw in name_lower for kw in keywords):
                    return printer['name']
            return printers[-1]['name'] if printers else None
        except Exception as e:
            logger.error(f"获取标签打印机失败: {e}")
            return None

// 文件：utils\rate_limiter.py
import logging
import time
from collections import OrderedDict
from threading import Lock
from typing import Any, Dict, Optional
logger = logging.getLogger(__name__)
class _InMemoryRateLimiter:
    def __init__(self, max_requests: int = 100, window_seconds: int = 60):
        self._max_requests = max_requests
        self._window_seconds = window_seconds
        self._requests: OrderedDict[str, list] = OrderedDict()
        self._lock = Lock()
```

```
    def _clean_old(self, key: str) -> None:
        now = time.time()
        cutoff = now - self._window_seconds
        self._requests[key] = [t for t in self._requests.get(key, []) if t > cutoff]
        if not self._requests[key]:
            del self._requests[key]
    def is_allowed(self, key: str) -> bool:
        with self._lock:
            self._clean_old(key)
            now = time.time()
            if key not in self._requests:
                self._requests[key] = []
            if len(self._requests[key]) < self._max_requests:
                self._requests[key].append(now)
                return True
            return False
    def get_remaining(self, key: str) -> int:
        with self._lock:
            self._clean_old(key)
            return max(0, self._max_requests - len(self._requests.get(key, [])))
    def get_reset_time(self, key: str) -> Optional[float]:
        with self._lock:
            self._clean_old(key)
            if key in self._requests and self._requests[key]:
                return self._requests[key][0] + self._window_seconds
            return None
class _CircuitBreaker:
    def __init__(
        self,
        failure_threshold: int = 5,
        recovery_timeout: int = 60,
        expected_exception: type = Exception
    ):
        self._failure_threshold = failure_threshold
        self._recovery_timeout = recovery_timeout
        self._expected_exception = expected_exception
        self._failure_count = 0
        self._last_failure_time: Optional[float] = None
        self._state = "closed"
        self._lock = Lock()
    @property
    def state(self) -> str:
        with self._lock:
            if self._state == "open":
                if self._last_failure_time and time.time() - self._last_failure_time > self._recovery_timeout:
                    self._state = "half-open"
                    logger.info("Circuit breaker transitioning to half-open")
            return self._state
    def call(self, func, *args, **kwargs):
        if self.state == "open":
```

```
            raise Exception("Circuit breaker is open")
        try:
            result = func(*args, **kwargs)
            with self._lock:
                if self._state == "half-open":
                    self._state = "closed"
                    self._failure_count = 0
                    logger.info("Circuit breaker closed after successful call")
            return result
        except self._expected_exception as e:
            with self._lock:
                self._failure_count += 1
                self._last_failure_time = time.time()
                if self._failure_count >= self._failure_threshold:
                    self._state = "open"
                    logger.warning(f"Circuit breaker opened after {self._failure_count} failures")
            raise e
    def reset(self) -> None:
        with self._lock:
            self._state = "closed"
            self._failure_count = 0
            self._last_failure_time = None
_rate_limiters: Dict[str, _InMemoryRateLimiter] = {}
_circuit_breakers: Dict[str, _CircuitBreaker] = {}
_limiter_lock = Lock()
def get_rate_limiter(name: str, max_requests: int = 100, window_seconds: int = 60) -> _InMemoryRateLimiter:
    with _limiter_lock:
        if name not in _rate_limiters:
            _rate_limiters[name] = _InMemoryRateLimiter(max_requests, window_seconds)
        return _rate_limiters[name]
def check_rate_limit(
    user_id: str,
    endpoint: str,
    max_requests: int = 100,
    window_seconds: int = 60
) -> Dict[str, Any]:
    key = f"{endpoint}:{user_id}"
    limiter = get_rate_limiter(endpoint, max_requests, window_seconds)
    if limiter.is_allowed(key):
        return {
            "allowed": True,
            "remaining": limiter.get_remaining(key),
            "reset_time": limiter.get_reset_time(key),
            "retry_after": None
        }
    else:
        reset_time = limiter.get_reset_time(key)
        retry_after = int(reset_time - time.time()) if reset_time else window_seconds
        return {
            "allowed": False,
```

```
        "remaining": 0,
        "reset_time": reset_time,
        "retry_after": retry_after
    }
def get_circuit_breaker(
  name: str,
  failure_threshold: int = 5,
  recovery_timeout: int = 60
) -> _CircuitBreaker:
  with _limiter_lock:
    if name not in _circuit_breakers:
      _circuit_breakers[name] = _CircuitBreaker(failure_threshold, recovery_timeout)
    return _circuit_breakers[name]
def reset_circuit_breaker(name: str) -> None:
  with _limiter_lock:
    if name in _circuit_breakers:
      _circuit_breakers[name].reset()

// 文件：utils\retry.py
import logging
from functools import wraps
from tenacity import (
  after_log,
  before_sleep_log,
  retry,
  retry_if_exception_type,
  stop_after_attempt,
  wait_exponential,
)
logger = logging.getLogger(__name__)
def retry_on_exception(
  max_attempts: int = 3,
  initial_wait: float = 1.0,
  max_wait: float = 30.0,
  multiplier: float = 2.0,
  exceptions: tuple = (Exception,)
):
  return retry(
    stop=stop_after_attempt(max_attempts),
    wait=wait_exponential(
      multiplier=multiplier,
      min=initial_wait,
      max=max_wait
    ),
    retry=retry_if_exception_type(exceptions),
    before_sleep=before_sleep_log(logger, logging.WARNING),
    after=after_log(logger, logging.INFO),
    reraise=True
  )
def retry_ai_service(
```

```
    max_attempts: int = 3,
    initial_wait: float = 1.0,
    max_wait: float = 30.0
):
    return retry_on_exception(
        max_attempts=max_attempts,
        initial_wait=initial_wait,
        max_wait=max_wait,
        multiplier=2.0,
        exceptions=(
            ConnectionError,
            TimeoutError,
            OSError,
        )
    )
def retry_network_operation(
    max_attempts: int = 3,
    initial_wait: float = 0.5,
    max_wait: float = 10.0
):
    return retry_on_exception(
        max_attempts=max_attempts,
        initial_wait=initial_wait,
        max_wait=max_wait,
        multiplier=2.0,
        exceptions=(
            ConnectionError,
            TimeoutError,
        )
    )

// 文件：utils\system_service.py
import logging
import os
import subprocess
import sys
from typing import Any, Dict, Optional
logger = logging.getLogger(__name__)
class SystemService:
    def __init__(self):
        self.app_name = "XCAGI"
        self.app_path = os.path.abspath(os.path.dirname(__file__))
    def get_startup_config(self) -> Dict[str, Any]:
        try:
            if sys.platform == "win32":
                import winreg
                try:
                    key_path = r"Software\Microsoft\Windows\CurrentVersion\Run"
                    key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, key_path, 0, winreg.KEY_READ)
                    try:
```

```
                value, _ = winreg.QueryValueEx(key, self.app_name)
                enabled = True
                startup_path = value
            except FileNotFoundError:
                enabled = False
                startup_path = None
            finally:
                winreg.CloseKey(key)
            return {
                "enabled": enabled,
                "app_path": self.app_path,
                "startup_path": startup_path,
                "platform": "windows"
            }
        except Exception as e:
            logger.error(f"获取开机自启配置失败：{e}")
            return {
                "enabled": False,
                "app_path": self.app_path,
                "error": str(e),
                "platform": "windows"
            }
    else:
        return {
            "enabled": False,
            "app_path": self.app_path,
            "platform": sys.platform,
            "message": "当前平台不支持开机自启"
        }
  except Exception as e:
    logger.exception(f"获取开机自启配置失败：{e}")
    return {
        "enabled": False,
        "app_path": self.app_path,
        "error": str(e)
    }
def enable_startup(self) -> Dict[str, Any]:
  try:
    if sys.platform == "win32":
        import winreg
        app_exe = sys.executable
        script_path = os.path.join(self.app_path, "..", "..", "run.py")
        script_path = os.path.abspath(script_path)
        startup_command = f'"{app_exe}" "{script_path}"'
        try:
            key_path = r"Software\Microsoft\Windows\CurrentVersion\Run"
            key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, key_path, 0, winreg.KEY_WRITE)
            winreg.SetValueEx(key, self.app_name, 0, winreg.REG_SZ, startup_command)
            winreg.CloseKey(key)
            logger.info(f"开机自启已启用：{startup_command}")
```

```
            return {
                "success": True,
                "message": "开机自启已启用",
                "command": startup_command
            }
        except Exception as e:
            logger.error(f"启用开机自启失败：{e}")
            return {
                "success": False,
                "message": f"启用失败：{str(e)}"
            }
    else:
        return {
            "success": False,
            "message": f"当前平台不支持开机自启：{sys.platform}"
        }
  except Exception as e:
    logger.exception(f"启用开机自启失败：{e}")
    return {
        "success": False,
        "message": f"启用失败：{str(e)}"
    }
def disable_startup(self) -> Dict[str, Any]:
  try:
    if sys.platform == "win32":
        import winreg
        try:
            key_path = r"Software\Microsoft\Windows\CurrentVersion\Run"
            key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, key_path, 0, winreg.KEY_WRITE)
            winreg.DeleteValue(key, self.app_name)
            winreg.CloseKey(key)
            logger.info("开机自启已禁用")
            return {
                "success": True,
                "message": "开机自启已禁用"
            }
        except FileNotFoundError:
            return {
                "success": True,
                "message": "开机自启原本就未启用"
            }
        except Exception as e:
            logger.error(f"禁用开机自启失败：{e}")
            return {
                "success": False,
                "message": f"禁用失败：{str(e)}"
            }
    else:
        return {
            "success": False,
```

```
                "message": f"当前平台不支持开机自启：{sys.platform}"
            }
        except Exception as e:
            logger.exception(f"禁用开机自启失败：{e}")
            return {
                "success": False,
                "message": f"禁用失败：{str(e)}"
            }
    def get_system_info(self) -> Dict[str, Any]:
        try:
            import platform
            return {
                "platform": sys.platform,
                "platform_version": platform.version(),
                "python_version": sys.version,
                "app_path": self.app_path,
                "working_directory": os.getcwd(),
                "executable": sys.executable
            }
        except Exception as e:
            logger.exception(f"获取系统信息失败：{e}")
            return {
                "error": str(e)
            }
    def get_printer_config(self) -> Dict[str, Any]:
        try:
            from app.infrastructure.printing.printer_adapter import PrinterAdapter
            printer_service = PrinterAdapter()
            try:
                printers = printer_service.list_printers()
                default_printer = printer_service.get_default_printer()
                return {
                    "success": True,
                    "printers": printers,
                    "default_printer": default_printer
                }
            except Exception as e:
                logger.error(f"获取打印机配置失败：{e}")
                return {
                    "success": False,
                    "message": f"获取打印机配置失败：{str(e)}",
                    "printers": [],
                    "default_printer": None
                }
        except Exception as e:
            logger.exception(f"获取打印机配置失败：{e}")
            return {
                "success": False,
                "message": f"获取打印机配置失败：{str(e)}"
            }
```

```
    def set_default_printer(self, printer_name: str) -> Dict[str, Any]:
        try:
            from app.infrastructure.printing.printer_adapter import PrinterAdapter
            printer_service = PrinterAdapter()
            try:
                success = printer_service.set_default_printer(printer_name)
                if success:
                    return {
                        "success": True,
                        "message": f"已设置默认打印机：{printer_name}"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"设置默认打印机失败：{printer_name}"
                    }
            except Exception as e:
                logger.error(f"设置默认打印机失败：{e}")
                return {
                    "success": False,
                    "message": f"设置失败：{str(e)}"
                }
        except Exception as e:
            logger.exception(f"设置默认打印机失败：{e}")
            return {
                "success": False,
                "message": f"设置失败：{str(e)}"
            }
def get_system_service() -> SystemService:
    return SystemService()

// 文件：utils\task_context.py
from __future__ import annotations
import time
from typing import Any, Dict, Optional
class TaskContextService:
    def __init__(self) -> None:
        self._store: Dict[str, Dict[str, Any]] = {}
        self._last_customers: Dict[str, Dict[str, Any]] = {}
    def get(self, user_id: str) -> Optional[Dict[str, Any]]:
        return self._store.get(user_id)
    def set(self, user_id: str, plan: Dict[str, Any]) -> None:
        payload = dict(plan or {})
        payload["updated_at"] = time.time()
        self._store[user_id] = payload
    def clear(self, user_id: str) -> None:
        self._store.pop(user_id, None)
    def set_last_customer(self, user_id: str, customer_data: Dict[str, Any]) -> None:
        payload = dict(customer_data or {})
        payload["updated_at"] = time.time()
```

```
        self._last_customers[user_id] = payload
    def get_last_customer(self, user_id: str) -> Optional[Dict[str, Any]]:
        return self._last_customers.get(user_id)
    def cleanup(self, max_age_seconds: int = 1800) -> int:
        now = time.time()
        expired = []
        for uid, item in self._store.items():
            if now - float(item.get("updated_at", 0)) > max_age_seconds:
                expired.append(uid)
        for uid in expired:
            self._store.pop(uid, None)
        expired_customers = []
        for uid, item in self._last_customers.items():
            if now - float(item.get("updated_at", 0)) > max_age_seconds:
                expired_customers.append(uid)
        for uid in expired_customers:
            self._last_customers.pop(uid, None)
        return len(expired) + len(expired_customers)
_task_context_service: Optional[TaskContextService] = None
def get_task_context_service() -> TaskContextService:
    global _task_context_service
    if _task_context_service is None:
        _task_context_service = TaskContextService()
    return _task_context_service

// 文件：utils\user_memory.py
import hashlib
import json
import logging
import os
import time
from collections import defaultdict
from dataclasses import asdict, dataclass, field
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional
logger = logging.getLogger(__name__)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
MEMORY_DIR = os.path.join(BASE_DIR, "user_memory")
JSON_MEMORY_PATH = os.path.join(MEMORY_DIR, "memory_store.json")
MAX_FEEDBACK_HISTORY = 100
MAX_FREQUENT_ACTIONS = 20
MAX_CONTEXT_SUMMARIES = 10
@dataclass
class ActionPattern:
    pattern: str
    intent: str
    slots: Dict[str, Any]
    frequency: int = 1
    last_used: str = field(default_factory=lambda: datetime.now().isoformat())
```

```
    confidence: float = 0.5
    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'ActionPattern':
        return cls(**data)
@dataclass
class FeedbackRecord:
    timestamp: str
    message: str
    recognized_intent: str
    user_feedback: str
    corrected_intent: Optional[str] = None
    slots: Dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'FeedbackRecord':
        return cls(**data)
@dataclass
class ContextSummary:
    timestamp: str
    intent: str
    slots: Dict[str, Any]
    message: str
    turn_count: int = 1
    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'ContextSummary':
        return cls(**data)
@dataclass
class UserMemory:
    user_id: str
    preferences: Dict[str, Any] = field(default_factory=dict)
    frequent_actions: List[Dict[str, Any]] = field(default_factory=list)
    historical_contexts: List[Dict[str, Any]] = field(default_factory=list)
    feedback_history: List[Dict[str, Any]] = field(default_factory=list)
    updated_at: str = field(default_factory=lambda: datetime.now().isoformat())
    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'UserMemory':
        return cls(**data)
class UserMemoryStore:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
```

```
        return cls._instance
    def __init__(self, storage_type: str = "json"):
        if hasattr(self, "_initialized") and self._initialized:
            return
        self.storage_type = storage_type
        self._memory_cache: Dict[str, UserMemory] = {}
        self._cache_dirty: Dict[str, bool] = {}
        self._load_all_memories()
        self._initialized = True
    def _load_all_memories(self) -> None:
        if self.storage_type == "json" and os.path.exists(JSON_MEMORY_PATH):
            try:
                with open(JSON_MEMORY_PATH, "r", encoding="utf-8") as f:
                    data = json.load(f)
                    for user_id, memory_data in data.items():
                        self._memory_cache[user_id] = UserMemory.from_dict(memory_data)
                logger.info(f"从 {JSON_MEMORY_PATH} 加载了 {len(self._memory_cache)} 个用户记忆")
            except Exception as e:
                logger.error(f"加载用户记忆失败: {e}")
                self._memory_cache = {}
    def _save_all_memories(self) -> None:
        if self.storage_type != "json":
            return
        try:
            os.makedirs(MEMORY_DIR, exist_ok=True)
            data = {user_id: memory.to_dict() for user_id, memory in self._memory_cache.items()}
            with open(JSON_MEMORY_PATH, "w", encoding="utf-8") as f:
                json.dump(data, f, ensure_ascii=False, indent=2)
            logger.debug(f"已保存 {len(self._memory_cache)} 个用户记忆到 {JSON_MEMORY_PATH}")
        except Exception as e:
            logger.error(f"保存用户记忆失败: {e}")
    def get_memory(self, user_id: str) -> Optional[UserMemory]:
        if user_id not in self._memory_cache:
            self._memory_cache[user_id] = UserMemory(user_id=user_id)
        return self._memory_cache[user_id]
    def save_memory(self, user_id: str, memory: UserMemory) -> None:
        memory.updated_at = datetime.now().isoformat()
        self._memory_cache[user_id] = memory
        self._cache_dirty[user_id] = True
        if self._should_persist():
            self._save_all_memories()
            self._cache_dirty[user_id] = False
    def _should_persist(self) -> bool:
        return any(self._cache_dirty.values())
class UserMemoryService:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
```

```
        return cls._instance
    def __init__(self, storage_type: str = "json"):
        if hasattr(self, "_initialized") and self._initialized:
            return
        self._store = UserMemoryStore(storage_type=storage_type)
        self._initialized = True
        logger.info("用户记忆服务已初始化")
    def add_preference(self, user_id: str, key: str, value: Any) -> None:
        memory = self._store.get_memory(user_id)
        if memory is None:
            memory = UserMemory(user_id=user_id)
        memory.preferences[key] = {
            "value": value,
            "updated_at": datetime.now().isoformat(),
            "count": memory.preferences.get(key, {}).get("count", 0) + 1
        }
        self._store.save_memory(user_id, memory)
        logger.debug(f"用户 {user_id} 偏好已更新: {key} = {value}")
    def get_preference(self, user_id: str, key: str, default: Any = None) -> Any:
        memory = self._store.get_memory(user_id)
        if memory and key in memory.preferences:
            return memory.preferences[key].get("value", default)
        return default
    def get_all_preferences(self, user_id: str) -> Dict[str, Any]:
        memory = self._store.get_memory(user_id)
        if memory:
            return {k: v.get("value") for k, v in memory.preferences.items()}
        return {}
    def record_action(
        self,
        user_id: str,
        intent: str,
        slots: Dict[str, Any],
        message: str = ""
    ) -> None:
        memory = self._store.get_memory(user_id)
        if memory is None:
            memory = UserMemory(user_id=user_id)
        pattern_key = self._make_pattern_key(intent, slots)
        existing_pattern = None
        pattern_idx = -1
        for idx, action in enumerate(memory.frequent_actions):
            if action.get("pattern") == pattern_key:
                existing_pattern = action
                pattern_idx = idx
                break
        if existing_pattern:
            existing_pattern["frequency"] += 1
            existing_pattern["last_used"] = datetime.now().isoformat()
            existing_pattern["confidence"] = min(0.99, existing_pattern["confidence"] + 0.05)
```

```
            memory.frequent_actions[pattern_idx] = existing_pattern
        else:
            new_pattern = ActionPattern(
                pattern=pattern_key,
                intent=intent,
                slots=slots,
                frequency=1,
                last_used=datetime.now().isoformat(),
                confidence=0.5
            )
            memory.frequent_actions.insert(0, new_pattern.to_dict())
        memory.frequent_actions.sort(key=lambda x: x.get("frequency", 0), reverse=True)
        memory.frequent_actions = memory.frequent_actions[:MAX_FREQUENT_ACTIONS]
        self._save_context_summary(memory, intent, slots, message)
        self._store.save_memory(user_id, memory)
        logger.debug(f"用户 {user_id} 操作已记录: intent={intent}, slots={slots}")
    def _make_pattern_key(self, intent: str, slots: Dict[str, Any]) -> str:
        key_parts = [intent]
        important_slots = ["unit_name", "product_name", "model_number"]
        for slot_key in important_slots:
            if slot_key in slots and slots[slot_key]:
                key_parts.append(f"{slot_key}={slots[slot_key]}")
        key_str = "|".join(key_parts)
        return hashlib.md5(key_str.encode()).hexdigest()[:16]
    def _save_context_summary(
        self,
        memory: UserMemory,
        intent: str,
        slots: Dict[str, Any],
        message: str
    ) -> None:
        summary = ContextSummary(
            timestamp=datetime.now().isoformat(),
            intent=intent,
            slots=slots,
            message=message[:100] if message else "",
            turn_count=1
        )
        memory.historical_contexts.insert(0, summary.to_dict())
        memory.historical_contexts = memory.historical_contexts[:MAX_CONTEXT_SUMMARIES]
    def get_recent_actions(
        self,
        user_id: str,
        limit: int = 5,
        intent_filter: Optional[str] = None
    ) -> List[Dict[str, Any]]:
        memory = self._store.get_memory(user_id)
        if not memory:
            return []
        actions = memory.frequent_actions
```

```
    if intent_filter:
        actions = [a for a in actions if a.get("intent") == intent_filter]
    return actions[:limit]
def get_similar_pattern(
    self,
    user_id: str,
    intent: str,
    slots: Dict[str, Any],
    threshold: float = 0.2
) -> Optional[Dict[str, Any]]:
    memory = self._store.get_memory(user_id)
    if not memory:
        return None
    best_match = None
    best_score = 0.0
    for action in memory.frequent_actions:
        if action.get("intent") != intent:
            continue
        score = self._calculate_similarity(slots, action.get("slots", {}))
        action_confidence = action.get("confidence", 0.5)
        if score >= 0.5:
            combined_score = score
        else:
            combined_score = score * action_confidence
        if combined_score > best_score and combined_score >= threshold:
            best_score = combined_score
            best_match = action
    if best_match:
        best_match["match_score"] = round(best_score, 3)
    return best_match
def _calculate_similarity(
    self,
    slots1: Dict[str, Any],
    slots2: Dict[str, Any]
) -> float:
    if not slots1 and not slots2:
        return 1.0
    important_keys = ["unit_name", "spec", "model_number", "quantity", "product_name"]
    match_count = 0
    total_count = 0
    for key in important_keys:
        v1 = slots1.get(key, "") or slots1.get(key)
        v2 = slots2.get(key, "") or slots2.get(key)
        if v1 and v2:
            total_count += 1
            if str(v1) == str(v2):
                match_count += 1
    if total_count == 0:
        return 0.5
    return match_count / total_count
```

```
  def add_feedback(
      self,
      user_id: str,
      message: str,
      recognized_intent: str,
      feedback: str,
      corrected_intent: Optional[str] = None,
      slots: Optional[Dict[str, Any]] = None
  ) -> None:
      memory = self._store.get_memory(user_id)
      if memory is None:
          memory = UserMemory(user_id=user_id)
      record = FeedbackRecord(
          timestamp=datetime.now().isoformat(),
          message=message[:200] if message else "",
          recognized_intent=recognized_intent,
          user_feedback=feedback,
          corrected_intent=corrected_intent,
          slots=slots or {}
      )
      memory.feedback_history.insert(0, record.to_dict())
      memory.feedback_history = memory.feedback_history[:MAX_FEEDBACK_HISTORY]
      self._adjust_pattern_weights(memory, recognized_intent, corrected_intent, feedback)
      self._store.save_memory(user_id, memory)
      logger.debug(f"用户 {user_id} 反馈已记录: feedback={feedback}, recognized={recognized_intent}")
  def _adjust_pattern_weights(
      self,
      memory: UserMemory,
      recognized_intent: str,
      corrected_intent: Optional[str],
      feedback: str
  ) -> None:
      weight_delta = 0
      target_intent = recognized_intent
      if feedback == "confirmed":
          weight_delta = 0.1
      elif feedback == "negated":
          weight_delta = -0.15
      elif feedback == "corrected" and corrected_intent:
          weight_delta = -0.1
          target_intent = corrected_intent
      for action in memory.frequent_actions:
          if action.get("intent") == target_intent:
              new_confidence = action.get("confidence", 0.5) + weight_delta
              action["confidence"] = max(0.1, min(0.99, new_confidence))
  def get_feedback_stats(self, user_id: str) -> Dict[str, Any]:
      memory = self._store.get_memory(user_id)
      if not memory:
          return {"total": 0, "confirmed": 0, "negated": 0, "corrected": 0}
      feedback_counts = defaultdict(int)
```

```
    intent_error_rates = defaultdict(lambda: {"total": 0, "errors": 0})
    for record in memory.feedback_history:
        fb_type = record.get("user_feedback", "unknown")
        feedback_counts[fb_type] += 1
        recognized = record.get("recognized_intent", "")
        intent_error_rates[recognized]["total"] += 1
        if fb_type in ("negated", "corrected"):
            intent_error_rates[recognized]["errors"] += 1
    error_rates = {}
    for intent, stats in intent_error_rates.items():
        if stats["total"] >= 3:
            error_rates[intent] = round(stats["errors"] / stats["total"], 3)
    return {
        "total": len(memory.feedback_history),
        "confirmed": feedback_counts.get("confirmed", 0),
        "negated": feedback_counts.get("negated", 0),
        "corrected": feedback_counts.get("corrected", 0),
        "error_rates": error_rates
    }
def get_habit_suggestions(self, user_id: str) -> List[Dict[str, Any]]:
    memory = self._store.get_memory(user_id)
    if not memory:
        return []
    suggestions = []
    action_sequence = self._analyze_action_sequence(memory)
    for seq in action_sequence:
        if seq["confidence"] >= 0.8 and len(seq["actions"]) >= 2:
            suggestions.append({
                "type": "action_sequence",
                "actions": seq["actions"],
                "confidence": seq["confidence"],
                "suggestion": f"执行 {seq['actions'][0]} 后主动提示 {seq['actions'][1]}"
            })
    return suggestions
def _analyze_action_sequence(self, memory: UserMemory) -> List[Dict[str, Any]]:
    sequences = defaultdict(lambda: {"count": 0, "first_action": ""})
    for i in range(len(memory.historical_contexts) - 1):
        current = memory.historical_contexts[i]
        next_ctx = memory.historical_contexts[i + 1]
        seq_key = f"{current.get('intent')}->{next_ctx.get('intent')}"
        sequences[seq_key]["count"] += 1
        sequences[seq_key]["first_action"] = current.get("intent")
    result = []
    for seq_key, stats in sequences.items():
        if stats["count"] >= 2:
            actions = seq_key.split("->")
            result.append({
                "actions": actions,
                "confidence": min(0.95, stats["count"] * 0.15),
                "count": stats["count"]
```

```
            })
        return result
    def apply_preference_to_slots(
        self,
        user_id: str,
        intent: str,
        slots: Dict[str, Any]
    ) -> Dict[str, Any]:
        filled_slots = slots.copy()
        if "unit_name" not in filled_slots or not filled_slots["unit_name"]:
            favorite_customer = self.get_preference(user_id, "favorite_customer")
            if favorite_customer:
                filled_slots["unit_name"] = favorite_customer
        if "template" not in filled_slots:
            default_template = self.get_preference(user_id, "default_template")
            if default_template:
                filled_slots["template"] = default_template
        return filled_slots
    def get_memory_summary(self, user_id: str) -> Dict[str, Any]:
        memory = self._store.get_memory(user_id)
        if not memory:
            return {"has_memory": False}
        return {
            "has_memory": True,
            "preference_count": len(memory.preferences),
            "action_count": len(memory.frequent_actions),
            "feedback_count": len(memory.feedback_history),
            "last_updated": memory.updated_at,
            "top_intents": [a.get("intent") for a in memory.frequent_actions[:3]]
        }
_user_memory_service: Optional[UserMemoryService] = None
def get_user_memory_service() -> UserMemoryService:
    global _user_memory_service
    if _user_memory_service is None:
        _user_memory_service = UserMemoryService()
    return _user_memory_service
def reset_user_memory_service() -> None:
    global _user_memory_service
    _user_memory_service = None

// 文件：utils\__init__.py
from app.utils.circuit_breaker import CircuitBreakerOpen, circuit_breaker, get_circuit_breaker
from app.utils.logger import get_logger, log_operation, setup_structured_logging
from app.utils.metrics import (
    init_metrics,
    metrics_endpoint,
    track_ai_request,
    track_request_duration,
)
from app.utils.retry import retry_ai_service, retry_network_operation, retry_on_exception
```